

EXPERIMENT 2

Shravanya Andhale

D20A-02

AIM: Create a Blockchain using Python

THEORY:

Blockchain is a distributed and immutable digital ledger used to store data in a sequence of interconnected blocks. Each block contains information such as a timestamp, transactions, a cryptographic hash, and a reference to the previous block. This linking of blocks helps maintain the integrity, transparency, and security of the stored data. Once information is added to a Blockchain and the block is accepted, modifying the information becomes difficult because any change in a block also changes its hash and affects the connection with the following blocks.

In this experiment, a simple Blockchain is implemented using Python and Flask. The Blockchain class is used to create and maintain blocks, handle transactions, perform mining, generate hashes, and validate the Blockchain. Flask is used to provide API endpoints through which the Blockchain can be accessed and tested using a web browser or Postman.

The major concepts implemented are:

1. **Block Creation:**

A block is the basic unit of a Blockchain. New blocks are created and added to the Blockchain in a sequential manner. Each block contains an index, timestamp, transactions, proof or nonce, and the hash of the previous block. The first block in the Blockchain is called the **genesis block**. The previous hash of the genesis block is generally set to 0 because it does not have a previous block. Every subsequent block stores the hash of the block immediately before it, creating a continuous chain.

2. **Transactions:**

Transactions represent the data or records that need to be stored in the Blockchain. In this experiment, transactions are maintained in a pending transaction list. A new transaction can be added to this list before the mining process begins. The block is mined only when the transaction list contains data and is not empty. Once the block is successfully mined, the transactions are added to the block and removed from the pending transaction list. This demonstrates how transaction data can be collected and permanently recorded in a Blockchain.

3. **SHA-256 Hashing:**

Cryptographic hashing is an important security feature of Blockchain technology. The SHA-256 hashing algorithm is used in this experiment to generate a unique fixed-length hash for the block data. The hash is generated using the information contained in the block. Even a small change in the block data produces a completely different hash. Therefore, hashing helps detect unauthorized changes and maintains the integrity of the information stored in the Blockchain. The hash of each block is also used by the next block to establish a secure connection between them.

4. **Proof-of-Work and Golden Nonce:**

Mining is the process of finding a suitable value called a **nonce** or proof that satisfies a specific cryptographic condition. In this experiment, the Proof-of-Work mechanism is used to search for the **golden nonce**. The cryptographic puzzle requires the generated block hash to have **three leading zeros (000)**. The program repeatedly changes the nonce and calculates the SHA-256 hash until a hash satisfying the required condition is obtained. The nonce that produces the required hash is considered the golden nonce. This process demonstrates the computational work involved in Blockchain mining.

5. **Mining:**

Mining is the process of adding a new block to the Blockchain after successfully solving the Proof-of-Work puzzle. Before mining, the program checks whether there are transactions available in the pending transaction list. If the transaction list is not empty, the program calculates the required nonce, generates the block hash, and creates a new block containing the transactions. After successful mining, the transactions are removed from the pending list. The newly created block is then added to the existing Blockchain.

6. **Blockchain Validation:**

Blockchain validation is used to verify the integrity of the complete chain. The Blockchain is validated by checking whether the previous hash stored in each block matches the actual hash of the previous block. The Proof-of-Work condition is also checked to ensure that each mined block satisfies the required cryptographic puzzle. If any information in a block is modified, its hash changes and the connection with the next block becomes invalid. Therefore, the validation process can detect whether the Blockchain has been tampered with or remains unchanged.

7. **Flask API:**

Flask is a lightweight Python web framework used to create the API for this Blockchain application. Different API endpoints are provided to interact with the Blockchain. The endpoints can be accessed using a web browser or Postman. The `/mine_block` endpoint is used to mine a new block, the `/get_chain` endpoint is used to display the complete Blockchain, and the `/is_valid` endpoint is used to check whether the Blockchain is valid. An additional transaction endpoint can be used to add new transactions to the pending transaction list.

8. **Immutability and Security:**

One of the main characteristics of Blockchain is immutability. Once a block is added to the chain, changing its contents will change its cryptographic hash. Since every following block contains the hash of the previous block, the modification can be detected during validation. The use of SHA-256 hashing and Proof-of-Work provides security and makes unauthorized modification of Blockchain data difficult.

The Blockchain implemented in this experiment is a simplified model intended to demonstrate the basic working principles of Blockchain technology. It does not contain all the features of a real-world Blockchain network, such as multiple distributed nodes, wallets, digital signatures, peer-to-peer communication, and advanced consensus mechanisms. However, it provides a practical understanding of how transactions are created, how blocks are mined, how hashes connect blocks, and how the integrity of the Blockchain can be verified.

CODE AND OUTPUT:

```
PS C:\Users\Shravanya> python --version
Python 3.10.0
PS C:\Users\Shravanya> cd "C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP2"
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP2> dir

Directory: C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP2


Mode                LastWriteTime         Length Name
----                -
-a----             02-09-2026    11:55           6009 blockchain.py

PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP2> python -m pip install flask==2.2.5
Collecting flask==2.2.5
  Downloading Flask-2.2.5-py3-none-any.whl (101 kB)
    |#####| 101 kB 3.3 MB/s
Requirement already satisfied: itsdangerous>=2.0 in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from flask==2.2.5) (2.2.0)
Requirement already satisfied: Werkzeug>=2.2.2 in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from flask==2.2.5) (2.3.8)
Requirement already satisfied: Jinja2>=3.0 in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from flask==2.2.5) (3.1.6)
Requirement already satisfied: click>=8.0 in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from flask==2.2.5) (8.2.1)
Requirement already satisfied: colorama in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from click>=8.0->flask==2.2.5) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\shravanya\appdata\local\programs\python\python310\lib\site-packages (from Jinja2>=3.0->flask==2.2.5) (2.1.2)
PS C:\Users\Shravanya\Desktop\BLOCKCHAIN_LAB\EXP2> python blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.180.253:5000
Press CTRL+C to quit
```

127.0.0.1:5000

Pretty print ☐

```
{"created_by": "Shravanya", "message": "Welcome to my blockchain project!", "project": "Simple Blockchain"}
```

127.0.0.1:5000/get_chain

Pretty print ☐

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-02 11:57:25.960975"
    },
    {
      "index": 2,
      "previous_hash": "7f1698257096a7326aab2921aafed6b2740ecef28079f6fcb5c82d07f550e23",
      "proof": 533,
      "timestamp": "2026-09-02 11:58:26.612591"
    },
    {
      "index": 3,
      "previous_hash": "eb0e1d402d4e801b49cc39de54c3e4afbc75d3f4e8db30709685581c0f8ec245",
      "proof": 45293,
      "timestamp": "2026-09-02 11:58:26.766600"
    }
  ],
  "length": 3
}
```

127.0.0.1:5000/mine_block

Pretty print ☐

```
{
  "index": 4,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "0e8aba6831a011108e7fa1abff95eedf3835193c09626225a3008bea6b22a207",
  "proof": 21391,
  "timestamp": "2026-09-02 11:59:10.060657"
}
```

```
← → ↻ ⓘ 127.0.0.1:5000/get_chain
Pretty print ☐
{"chain":[{"index":1,"previous_hash":"0","proof":1,"timestamp":"2026-09-02 11:57:25.960975"},
{"index":2,"previous_hash":"7f1698257096a7326aab2921aafed6b2740ecef28079f6fcfb5c82d07f550e23","proof":533,"timestamp":"2026-09-02 11:58:26.612591"},
{"index":3,"previous_hash":"eb0e1d402d4e801b49cc39de54c3e4afbc75d3f4e8db30709685581c0f8ec245","proof":45293,"timestamp":"2026-09-02 11:58:26.766600"},
{"index":4,"previous_hash":"0e8aba6831a011108e7fa1abff95eedf3835193c09626225a3008bea6b22a207","proof":21391,"timestamp":"2026-09-02 11:59:10.060657"},
{"index":5,"previous_hash":"22ca6f2098b80b6bb8a4668e32c354f5d23b2a90e07d3ec5c6ed4e6119e9923b","proof":8018,"timestamp":"2026-09-02 11:59:10.117801"}], "length":5}
```

```
← → ↻ ⓘ 127.0.0.1:5000/is_valid
Pretty print ☐
{"message":"All good. The Blockchain is valid."}
```

CONCLUSION:

The Blockchain was successfully implemented using Python and Flask. The experiment demonstrated block creation, transaction handling, SHA-256 hashing, Proof-of-Work mining using a golden nonce, and Blockchain validation. The Blockchain can be interacted with through Flask API endpoints and tested using Postman. This experiment helped in understanding how cryptographic hashing and mining maintain the integrity and security of data in a Blockchain.